SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1701**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I GIFTSON KEVIN MATTHEW.Y (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIFTSON KEVIN MATTHEW.Y

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: AI-Powered Drones for Emergency Medicine Delivery

Introduction

AI-powered drones deliver emergency medicines to flood-affected areas where paths may be blocked and weather conditions change frequently. The drone must find a route that minimizes travel time and risk. Greedy Best-First Search and A* Search can help solve this problem.

i. Greedy Best-First Search: Working, strengths, and weaknesses

Greedy Best-First Search selects the node that appears closest to the destination using a heuristic function.

[f(n)=h(n)]

Here, (h(n)) is the estimated remaining distance to the goal.

Working: If a drone has routes of 2 km, 3 km, and 4 km, it initially prefers the 2 km route. However, if that route is blocked, it must select another available path.

Strengths:

- Makes decisions quickly.
- Uses heuristic information to guide the search.
- Suitable for finding promising routes during emergencies.

Weaknesses:

- May choose a short but dangerous route.
- Does not guarantee the lowest total travel cost.
- Inaccurate information may lead to poor decisions.
- Blocked paths require replanning.

ii. A* Search and its cost-heuristic balance

A* considers both the cost already travelled and the estimated remaining cost.

[boxed{f(n)=g(n)+h(n)}]

Where (g(n)) is the actual cost from the start and (h(n)) is the estimated cost to the goal.

For example:

Route	(g(n))	(h(n))	(f(n))
A	8	2	10
B	4	5	9

A* selects Route B because its total estimated cost is lower.

It can provide an optimal route when costs are non-negative and the heuristic and implementation satisfy the required conditions. However, it may use more memory and computation than Greedy Best-First Search.

iii. Impact of heuristic accuracy

In Greedy Best-First Search, the heuristic directly determines which node is explored next. An inaccurate estimate may lead the drone toward blocked or unsafe paths.

In A*, an inaccurate heuristic can increase computation and affect route quality. If the heuristic overestimates the true remaining cost, optimality may not be guaranteed.

Therefore, reliable estimates of distance, travel time, and weather conditions improve search performance.

iv. Impact of dynamic changes

When paths become blocked or weather changes, the drone must update its map and reconsider the route. This increases computation and may invalidate its existing plan.

Greedy Best-First Search may quickly find an alternative but not necessarily the best one. A* considers the total estimated route cost but may require expensive replanning. Incremental algorithms such as D* Lite can help reuse previous search information.

v. Recommendation

A Search with dynamic replanning* is the most suitable general approach because it considers both the cost already travelled and the estimated remaining cost.

The drone should update its map, account for weather and safety risks, and replan when a route becomes unsafe. For frequently changing conditions, D* Lite can improve replanning efficiency.

Conclusion: A* offers a better balance of route quality and efficiency than Greedy Best-First Search. Safety constraints must always take priority over selecting a short route.

Question 2: Smart City Traffic Signal Optimization

Introduction

A smart city uses AI to adjust traffic signals to reduce waiting time, fuel consumption, and congestion. Since traffic conditions change continuously and hundreds of intersections are involved, efficient optimization techniques are required.

Formulate the optimization problem and explain why traditional search is inefficient.

The system must select suitable green-light durations and signal sequences while satisfying safety rules.

A possible objective function is:

[boxed{J=w_1W+w_2F+w_3C}]

Where (W) is waiting time, (F) is fuel consumption, (C) is congestion, and (w_1,w_2,w_3) are their respective weights.

The objective is to minimize (J) while respecting traffic safety and signal-timing constraints.

Traditional search can be inefficient because:

- Hundreds of intersections create a huge search space.
- Traffic conditions change dynamically.
- Evaluating every possible configuration requires considerable time.
- Local search may get stuck in a locally optimal solution.

i. Hill Climbing: Working and limitations

Hill Climbing starts with an initial signal configuration and repeatedly selects a neighbouring configuration that improves the objective.

Working:

1. Choose initial traffic timings.
2. Calculate waiting time, fuel consumption, and congestion.
3. Change one or more signal timings.
4. Accept a better configuration.
5. Repeat until no better neighbour is found.

Example: If increasing a green-light duration from 30 to 35 seconds reduces congestion, the system accepts the change.

Limitations: Hill Climbing may stop at a local optimum, struggle on plateaus, fail to coordinate multiple intersections, and depend heavily on its initial solution.

ii. Local maxima, plateaus, and ridges

- Local maximum: A solution that is better than nearby alternatives but not the best overall. For example, improving one intersection may prevent the system from discovering a better city-wide timing plan.
- Plateau: A region where several timing configurations produce nearly identical results. The algorithm may not know which direction to explore.
- Ridge: Improvement requires changing multiple variables together. For example, two intersections may need synchronized green-light timings to reduce congestion.

These problems can prevent Hill Climbing from finding better solutions.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing: Sometimes accepts a worse solution to escape a local optimum.

[P=e^{-\Delta J/T}]

Here,(\Delta J) is the increase in cost and (T) is the temperature. Higher temperatures allow more exploration, while lower temperatures favour better solutions.

Genetic Algorithm: Maintains a population of candidate signal plans and improves them through selection, crossover, and mutation. It can explore many configurations and combine useful timing patterns.

Simulated Annealing is simpler for single-solution exploration, while Genetic Algorithms are useful for searching large, complex solution spaces. Neither guarantees a globally optimal result.

iv. Recommendation

A hybrid Genetic Algorithm with fast local optimization is suitable for a large smart city. The Genetic Algorithm can explore coordinated signal plans, while local optimization adjusts timings according to real-time traffic.

The system must continuously monitor traffic and enforce safety constraints.

Conclusion: Combining broad search with fast local adjustments provides a good balance of scalability, solution quality, and adaptability.

Question 3: Autonomous Rover Exploring Mars

Introduction

A Mars rover must explore unknown terrain, collect samples, and avoid rocks and craters. Since its sensors provide incomplete information, the rover must learn about its surroundings while moving. Therefore, online search is more suitable than planning an entire route in advance.

i. Why an online search agent is required

Offline search generally plans using an available model before executing the solution. Online search selects actions while discovering the environment.

For example, if the rover detects an unexpected crater, it updates its map and finds an alternative route.

An online search agent is suitable because it can:

- Discover unknown terrain.
- Respond to newly detected obstacles.
- Update its knowledge after every movement.
- Replan when the existing route becomes unsafe.

ii. Challenges of partial observability and dynamic environments

Partial observability means the rover cannot observe the complete environment. Rocks may hide obstacles, sensors may contain errors, and terrain conditions may be uncertain.

Dynamic conditions include changing lighting, dust, wheel traction, and newly discovered hazards.

These challenges require the rover to maintain an estimated state, combine sensor information, avoid risky paths, and replan when necessary. It must also consider battery capacity and mission time.

iii. How the rover builds and updates its internal model

The rover maintains an internal representation of its environment.

Working process:

1. Sensing: Collect camera and other sensor data.
2. Interpretation: Identify rocks, craters, slopes, and safe areas.
3. Map updating: Record known obstacles and unexplored regions.
4. Planning: Select a route based on distance, energy, and risk.
5. Execution: Move, gather new observations, and update the model again.

The model includes the rover's estimated position, terrain map, travel costs, uncertainty, and sample locations.

For example, discovering loose soil increases the estimated cost of that route, allowing the rover to select a safer alternative.

iv. Comparison of online, uninformed, and informed search

Feature	**Uninformed search**	**Informed search**	**Online search**
Guidance	No heuristic	Uses a heuristic	Uses current observations
Knowledge	Available problem information	Problem information and estimates	Learns while acting
Example	BFS, DFS	A*	Exploration with replanning
Limitation	May explore many states	Depends on heuristic quality	Must act with incomplete information

Online search describes searching while interacting with the environment, whereas informed and uninformed search describe how search decisions are guided. An online rover can also use heuristic planning.

v. Recommendation

An online, uncertainty-aware search agent with heuristic planning is the most suitable approach. The rover should maintain an updated map, use A*-like planning to identify promising paths, assign higher costs to dangerous terrain, and replan when new obstacles are discovered. It should stop or choose a safer action when sensor information is unreliable.

Conclusion: Online search with mapping and heuristic planning allows the rover to adapt to unknown terrain, make safer decisions, and complete its exploration mission efficiently.

Question 4: University Examination Timetable Using CSP

Introduction

A university must schedule examinations without student clashes, while considering hall capacity, subject ordering, faculty availability, and special accommodations. A Constraint Satisfaction Problem (CSP) is suitable because it represents these requirements explicitly.

A CSP is defined as:

[boxed{text{CSP}=(X,D,C)}]

Where (X) represents variables, (D) represents domains, and (C) represents constraints.

i. Variables, domains, and constraints

1. Variables

- (T_e): time slot assigned to examination (e).
- (H_e): hall assigned to examination (e).
- (F_e): assigned invigilator, if required.

2. Domains

Domains contain the possible values for each variable. For example, an examination may be assigned to Slot 1, Slot 2, or Slot 3, and Hall A or Hall B.

3. Constraints

- No student clashes: Examinations sharing students must have different time slots.

[T_i\ne T_j]

- Hall availability: Two examinations at the same time cannot occupy the same hall.

[T_i=T_j\Rightarrow H_i\ne H_j]

- Subject ordering: Prerequisite examinations must be scheduled before dependent examinations.
- Faculty availability: Examinations must be assigned to available faculty.
- Special accommodations: Suitable halls and additional requirements must be provided.

The objective is to find assignments satisfying all hard constraints. Soft preferences, such as avoiding consecutive difficult examinations, can be optimized afterward.

ii. How backtracking search works

Backtracking Search assigns values to variables one by one and reverses an assignment when it violates a constraint.

Steps:

1. Select an unscheduled examination.
2. Assign a possible time and hall.
3. Check the constraints.
4. Keep the assignment if it is valid.
5. If no value works, backtrack and change an earlier assignment.
6. Continue until a valid timetable is found or all possibilities are exhausted.

Example: If Mathematics is scheduled in Slot 1, Physics cannot be scheduled in Slot 1 when the same students take both examinations. The algorithm tries another slot.

iii. How forward checking improves efficiency

Forward checking removes values from the domains of unassigned variables when those values conflict with an existing assignment.

For example, if Mathematics is assigned to Slot 1, the system removes Slot 1 from the domain of Physics when both examinations share students.

If Physics has only Slot 3 remaining, the system can assign that slot, provided all other constraints are satisfied. If a domain becomes empty, it immediately backtracks.

Forward checking reduces unnecessary search. Other useful strategies include:

- MRV: Select the variable with the fewest legal values.
- Degree heuristic: Select the variable that constrains the most other variables.
- Least Constraining Value: Try the value that leaves the most choices for other variables.

iv. Real-world challenges and recommended strategy

The main challenges are large numbers of examinations, student clashes, limited halls, faculty availability, changing registrations, and conflicting preferences.

Recommendation: Use backtracking with MRV and forward checking.

The system should define all hard constraints, schedule the most constrained examinations first, prune invalid choices early, and optimize soft preferences after finding a valid timetable. For large universities, an optimized constraint-programming solver can improve scalability.

Conclusion: CSP provides a systematic way to generate valid examination timetables. Backtracking and constraint propagation reduce unnecessary exploration and help the university handle complex scheduling requirements.

Question 5: Strategic Game AI Using Minimax and Alpha-Beta Pruning

Introduction

A real-time strategy game AI must compete against human players, evaluate possible moves, and make decisions within strict time limits. Minimax models the opponent's responses, while Alpha-Beta Pruning reduces unnecessary computation.

i. How Minimax models the problem

Minimax is an adversarial search algorithm in which:

- MAX selects the move with the highest evaluation.
- MIN selects the move with the lowest evaluation for MAX.

The algorithm represents possible moves as a game tree and evaluates the outcomes.

Example: Suppose Move A leads to opponent-response scores of 3 and 5, while Move B leads to scores of 2 and 9.

MIN selects the worst outcome for MAX in each branch:

$$A=\min(3,5)=3$$

$$B=\min(2,9)=2$$

MAX then chooses:

$$\max(3,2)=3$$

Therefore, the AI chooses Move A because its worst-case result is better.

Minimax is useful for strategic decision-making but can become expensive when the game tree is large.

ii. Computational challenges of large game trees

The approximate number of nodes in a full game tree is:

$$\boxed{N=\frac{b^{d+1}-1}{b-1}}$$

Where (b) is the average number of possible moves per state and (d) is the search depth.

For example, with 10 possible moves per state and a depth of 5 plies:

$$N=1+10+100+1000+10000+100000$$

$$N=111111$$

The main challenges are exponential growth, high memory usage, complex state evaluation, and strict decision deadlines. Therefore, exploring the complete game tree is usually impractical.

iii. How Alpha-Beta Pruning improves efficiency

Alpha-Beta Pruning improves Minimax by skipping branches that cannot affect the final decision.

- (alpha): Best value MAX can guarantee so far.
- (beta): Best value MIN can guarantee so far.

The pruning condition is:

[boxed{\alpha\geq\beta}]

Example:

1. Move A produces a Minimax value of 3.
2. MAX updates (alpha=3).
3. Move B reaches a MIN node whose first leaf has value 2.
4. MIN can guarantee a value no greater than 2, so (beta=2).
5. Since (alpha\geq\beta), the remaining branches under B can be pruned.

MAX already has a better choice through A, so those branches cannot change the final decision.

Benefits:

- Reduces the number of nodes evaluated.
- Allows deeper searches within the same time.
- Preserves the Minimax result for the same tree.
- Works best when promising moves are explored first.

With ideal move ordering, Alpha-Beta Pruning can improve time complexity from approximately (O(b^d)) to (O(b^{d/2})).

iv. Design an evaluation function

An evaluation function estimates the strategic value of a game state.

[boxed{E(s)=w_uU+w_rR+w_tT+w_bB+w_oO}]

Where:

- (U): Unit advantage.
- (R): Resource advantage.
- (T): Territory control.
- (B): Base strength.
- (O): Objective progress.
- (w_u,w_r,w_t,w_b,w_o): Weights assigned to each factor.

For example, if the AI has strong units, sufficient resources, and good territory control, the state may receive a positive score.

The weights should reflect the game's victory conditions. Unit strength helps evaluate combat ability, resources support future actions, territory gives strategic advantages, base strength measures defence, and objective progress indicates how close the player is to winning.

v. Recommendation for balancing optimality and real-time performance

Recommendation: Use Minimax with Alpha-Beta Pruning and iterative deepening.

The system should:

1. Prune branches that cannot affect the result.
2. Search progressively deeper using iterative deepening.
3. Examine promising moves first.
4. Use an evaluation function for non-terminal states.
5. Stop within the available time and return the best move from the last completed search.
6. Reuse previous results when possible through a transposition table.

This approach reduces unnecessary computation while maintaining good decision quality. However, a depth-limited search does not guarantee the globally best move for the entire game.

Conclusion: Minimax with Alpha-Beta Pruning, iterative deepening, and a suitable evaluation function provides an effective balance between strategic quality and real-time performance.